

Natural Language Processing - Ex4

Nurit Tolkowsky and Shaul Tolkowsky

1 MST Parser

1.4 Evaluation (UAS)

We trained the averaged perceptron on the first 90% of the `dependency_treebank` parsed sentences (3522 sentences), for 2 iterations with learning rate 1, traversing the training instances in a random order. Edge scores are the dot product of the weight vector with the word-bigram and POS-bigram feature function; inference uses the Chu-Liu/Edmonds algorithm (`cle_min`) applied to negated scores (minimum arborescence = maximum spanning tree). Evaluated on the last 10% of the corpus (392 sentences), the mean unlabeled attachment score (UAS) of the learned averaged weight vector is:

$$\text{UAS (averaged perceptron)} = 0.1768.$$

2 Attention-Based Parsing

2.2(b) UAS per BERT layer (`head_mode = "mean"`)

Using `get_attention_matrix`, `attn_to_arc_scores` (which negates the attention weights for `cle_min` and never creates an edge into ROOT, since ROOT can never be a dependent), and `cle_min`, we parsed every sentence in the test set. The mean UAS for each setting is:

Setting (<code>head_mode = mean</code>)	UAS
<code>layer = 0</code> (first BERT layer)	0.230
<code>layer = 5</code> (middle BERT layer)	0.138
<code>layer = 11</code> (last BERT layer)	0.120

Discussion - which setting works best and why. The first layer (`layer = 0`) gives the highest UAS. Because `head_mode="mean"` averages all heads, the score reflects each layer's average attention pattern: in the lowest layer this pattern is dominated by local, positional behaviour (words attend mostly to their immediate neighbours). Since a large fraction of dependency arcs connect nearby words, this surface locality lines up with the gold tree better than the more diffuse, semantically-oriented attention of the middle and upper layers, where the few genuinely syntactic heads are washed out by averaging.

This may seem to contradict the intuition stated in the exercise - that, since lower layers are closer to surface syntax and upper layers to semantics, the middle layers should be most useful for parsing. A likely reason it does not hold here is that we average all heads together (`head_mode="mean"`): if only a few individual heads track syntax, averaging dilutes them, so the score is dominated by each layer's overall attention pattern. In the lowest layer that pattern is strongly local, and since many dependencies are between nearby words, `layer = 0` ends up winning.

Side note: the orientation of the attention matrix. The matrix is directional: `attn[u,v]` is how much word u attends to word v . The task’s convention treats u as head and v as dependent, so we score arcs with `-attn[u,v]` (reported above). But a dependent often attends to its head rather than the reverse, so we also tried the transposed score `-attn[v,u]`:

layer	original <code>attn[u,v]</code>	transposed <code>attn[v,u]</code>
0	0.230	0.160
5	0.138	0.217
11	0.120	0.013

Under the transposed orientation the middle layer becomes the best (layer 5 at 0.217, above layers 0 and 11), recovering the expected trend that middle layers are the most syntactic, whereas in the original orientation the surface-local layer 0 wins. This suggests that the syntactically meaningful signal is a dependent attending to its head, and that layer 0’s high original score reflects positional locality rather than real syntax. We report the non-transposed orientation as our primary result, following the convention of the provided `get_attention_matrix`.

2.3 Comparison with the MST parser

Parser	UAS
Perceptron MST (averaged weights)	0.1768
BERT attention, best setting (<code>layer = 0</code>)	0.230

Data requirements. The perceptron needs labeled, dependency-annotated training data plus a hand-designed feature function, whereas the attention-based parser needs no task-specific training at all - it reuses attention weights from a model pre-trained only on masked language modeling.

Inference cost. The perceptron is far cheaper, scoring each candidate edge with a sparse dot product, while the attention parser requires a full BERT forward pass per sentence.

Accuracy. Here the best attention setting (0.230) slightly outperforms our 2-iteration perceptron (0.1768), which is notable given that BERT never saw a parse tree. Part of the perceptron’s low score is that its feature function is deliberately simple: every feature looks only at a pair - the two endpoints of a single edge (one word bigram and one POS bigram) - with no higher-order features (such as word triplets, head-dependent distance, or combined word+POS features) and no surrounding context. The word-bigram features are also extremely sparse - most word pairs seen at test time never appeared in training - so they barely generalize, and with only 2 training iterations the model stays underfit. BERT, by contrast, encodes each word using the whole sentence, so even its raw attention carries far richer structural information.

Why untrained attention correlates with syntax. Accurately predicting a masked word requires implicitly modeling grammatical relationships - which words govern or modify which - so some attention heads end up tracking head-dependent structure as a by-product of language modeling.